Lecture review

C++ is very similar to the C Language.

- For the input/output stream we use `<iostream>` library (in C it was `<stdio>`).
- For taking input and output we use `cout` and `cin` (in C it was `printf` and `scanf`).
  - `cout` uses insertion («) operator.
  - `cin` uses extraction (») operator.

Sample C++ Code:

```
#include <iostream>
using namespace std;
int main()
{
    int var = 0;

    cout << "Enter an Integer value: ";
    cin >> var;
    cout << "Value of var is : " << var;
    return 0;
}
```

Sample Run:

```
Enter an Integer value:  12
Value of var is :  12
```

1 ARRAYS

1.1 1-D ARRAY:

An array is a collection of a fixed number of elements of the same data type.

- To declare a 1D array, you need to specify the data type, name, and array size:

```
dataType arrayName [arraySize];
```

- Example declaration:

```
int numArray[5];
```

  - Data Type: Integers
  - Name: `numArray`
  - Size: 5

- To access array elements, you use the array name along with the index in subscript operator `[ ]`.

  `numArray[0], numArray[1], numArray[2], numArray[3], numArray[4].`

- Index of the array starts from 0, and the last index is always `size - 1`.

Example Code for 1-D Array:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int item[5]; // Declaring an array of size 5
    int sum = 0; // Variable to store the sum
    int counter;

    // Input step
    cout << "Enter five numbers: ";

    // First loop to input numbers and calculate the sum
    for (counter = 0; counter < 5; counter++)
    {
        cin >> item[counter]; // Takes input for each element of the array
        sum = sum + item[counter]; // Adds the input to the sum
    }
    // This loop runs 5 times. So, its time complexity is O(5), which simplifies to O(1).

    // Output the sum
    cout << endl;
    cout << "The sum of the numbers is: " << sum << endl;

    // Output numbers in reverse order
    cout << "The numbers in reverse order are: ";
    for (counter = 4; counter >= 0; counter--)
        cout << item[counter] << " ";
    // This loop also runs 5 times. Time complexity is O(5), which simplifies to O(1).

    cout << endl;
    return 0;
}
```

Sample Run:

```
Enter five numbers:  12 76 34 52 89
The sum of the numbers is:  263
The numbers in reverse order are:  89 52 34 76 12
```

1.2 2-D ARRAY:

A 2-D Array is a collection of fixed elements arranged in rows and columns.

- To declare a 2D array, specify the data type, name, and number of rows and columns:

```
dataType arrayName [rowSize] [columnSize];
```

- Example declaration:

```
int numArray[5][5];
```

  - Data Type: Integers
  - Name: `numArray`
  - Rows: 5
  - Columns: 5

- To access array elements, you use the array name along with the row and column indices in subscript operator `[ ][ ]`.

```
numArray[0][0], numArray[1][1], numArray[2][2], numArray[3][3], numArray[4][4].
```

- Index for rows and columns starts from zero, and the last index is `sizeofRow - 1` and `sizeofColumn - 1`.

Example Code for 2-D Array:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int item[3][3]; // Declaring a 3x3 array
    int sum = 0;    // Variable to store the sum of elements
    int row, col;   // Loop variables

    // Input prompt
    cout << "Enter array elements: " << endl;

    // Nested loop to input elements and calculate the sum
    for (row = 0; row < 3; row++)  // Outer loop runs 3 times (for rows)
    {
        for (col = 0; col < 3; col++)  // Inner loop runs 3 times for each row (for columns)
        {
            cin >> item[row][col];     // Input each element
            sum = sum + item[row][col]; // Add element to the sum
        }
        // Print the sum of the row
        cout << "The sum of row " << row << " : " << sum << endl;
    }
```

```
    cout << endl;
    return 0;
}
```

Sample Run:

```
Enter array elements:
12 76 34
The sum of row 0 :  122
52 89 48
The sum of row 1 :  189
22 63 99
The sum of row 2 :  184
```

2 POINTERS

A pointer is a variable whose content is a memory address.

2.1 Single Pointers:

- To declare a single pointer variable, you need to specify the data type, an asterisk symbol (*) and the pointer variable name:

```
dataType *ptrName;
```

- Example declaration:

```
int *ptr;
```

- The pointer variable holds the memory address of the variable of the same data type.
- To assign the memory address of a variable to a pointer variable, use the address-of operator (&):

```
ptr = &intVar;
```

- To access the value at the memory address, use the dereferencing operator (*):

```
int intVar2 = *ptr;
```

Example Code for Single Pointers:

```
#include <iostream>
using namespace std;
int main()
{
    int *p;
    int x = 37;
    cout << "Line 1: x = " << x << endl;
```

```
    p = &x;
    cout << "Line 3: *p = " << *p << ", x = " << x << endl;
    *p = 58;
    cout << "Line 5: *p = " << *p << ", x = " << x << endl;
    cout << "Line 6: Address of p = " << &p << endl;
    cout << "Line 7: Value of p = " << p << endl;
    cout << "Line 8: Value of the memory location pointed to by *p = " << *p << endl;
    cout << "Line 9: Address of x = " << &x << endl;
    cout << "Line 10: Value of x = " << x << endl;
    return 0;
}
```

Sample Run:

```
Line 1:  x = 37
Line 3:  *p = 37, x = 37
Line 5:  *p = 58, x = 58
Line 6:  Address of p = 006BFDF4
Line 7:  Value of p = 006BFDF0
Line 8:  Value of the memory location pointed to by *p = 58
Line 9:  Address of x = 006BFDF0
Line 10:  Value of x = 58
```

3 DYNAMIC VARIABLES

Variables created during the program execution are called dynamic variables.

- To create a dynamic variable, use the `new` operator:

```
new dataType;
```

- To create a dynamic array:

```
new dataType [size];
```

- Example declaration:

```
int *p = new int;

char *cArray = new char[5];
```

- To delete dynamically allocated memory, use the `delete` operator:

```
delete ptrVar;

delete [] ptrArray;
```

Example Code for Dynamic Variables:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int* intPtr;         // Pointer to hold an integer value
    char* charArray;     // Pointer to hold a dynamic character array
    int arraySize;       // Variable to store the size of the character array

    intPtr = new int;    // Allocating memory for one integer (O(1))
    cout << "Enter an Integer Value: ";
    cin >> *intPtr;      // Reading one integer (O(1))

    cout << "Enter the size of the Character Array: ";
    cin >> arraySize;    // Reading size of the array (O(1))

    charArray = new char[arraySize]; // Allocating dynamic memory for an array (O(arraySize))

    // Loop to take input for the character array
    for (int i = 0; i < arraySize; i++)  // Runs `arraySize` times
        cin >> charArray[i];             // Each input operation is O(1)
    // Total time complexity of this loop: O(arraySize)

    // Loop to print the character array
    for (int i = 0; i < arraySize; i++)  // Runs `arraySize` times
        cout << charArray[i];            // Each output operation is O(1)
    // Total time complexity of this loop: O(arraySize)

    return 0;
}
```

Sample Run:

```
Enter an Integer Value:  2
Enter the size of the Character Array:  2
a b
ab
```

4 STRUCTURES

A structure is a collection of fixed number of components, which can be of different types, and the components are accessed by name.

- Components of a structure are called members.
- To declare a structure, use the `struct` keyword:

```
struct structName { dataType1 varName1; ...  };
```

Example Code for Structure:

```cpp
#include <iostream>
using namespace std;

struct studentType
{
    string firstName;
    string lastName;
    char courseGrade;
    int courseScore;
    double GPA;
};

int main()
{
    studentType newStudent;
    cout << "Enter Details for the Student" << endl;
    cout << "Enter Student's First Name : ";
    cin >> newStudent.firstName;

    cout << "Enter Student's Last Name : ";
    cin >> newStudent.lastName;

    cout << "Enter Student's Course Grade : ";
    cin >> newStudent.courseGrade;

    cout << "Enter Student's Course Score : ";
    cin >> newStudent.courseScore;

    cout << "Enter Student's GPA : ";
    cin >> newStudent.GPA;

    cout << newStudent.firstName << endl;
    cout << newStudent.lastName << endl;
    cout << newStudent.courseGrade << endl;
    cout << newStudent.courseScore << endl;
    cout << newStudent.GPA << endl;
}
```

Sample Run:

```
Enter Details for the Student
Enter Student's First Name:  First_Name
Enter Student's Last Name:  Last_Name
Enter Student's Course Grade:  A
Enter Student's Course Score:  84
Enter Student's GPA: 2.0
First_Name
Last_Name
```

```
A
84
2
```

5 Encapsulation

Encapsulation is one of the fundamental concepts in object-oriented programming (OOP). It describes the idea of bundling data and methods that work on that data within one unit, e.g., a class in C++. This concept is also often used to hide the internal representation, or state, of an object from the outside. This is called information hiding.

Encapsulation can be achieved by making *instance variables* of the class `private` and the access to the members is then controlled by `public` *member functions* of the class.

For instance, the `Student` class below has three private instance variables `erpID`, `name`, and `cgpa`.

```
class Student {
private:
    int erpID;
    string name;
    float cgpa;
public:
    Student(int id, string n, float c) { // constructor
        erpID = id;
        name = n;
        cgpa = c;
    }
    void setCGPA(float c) {    // setter
        cgpa = c;
    }
    int getErpID() {           // getter
        return erpID;
    }
    string getName() {         // getter
        return name;
    }
    float getCGPA() {          // getter
        return cgpa;
    }
};
```

The public interface of the class includes a constructor that takes three parameters and initializes the instance variables `erpID`, `name`, and `cgpa`.

There is a public member functions `setCGPA()` to set the value of the instance variable `cgpa`. Such member functions are also called *setters* or *mutators*. Note that we have not provided setter functions for `erpID` and `name`. This is because we do not want to allow the user to change these values once they are initialized in the constructor.

It also has *getter* or *accessor* functions `getErpID()`, `getName()`, and `getCGPA()` to access the values of the instance variables.

Lab exercises

Exercise 1

Create a class called `Date` that includes three pieces of information as data members—a `day` (type `int`), a `month` (type `int`) and a `year` (type `int`).

Your class should have a constructor with three parameters that uses the parameters to initialize the three data members. For the purpose of this exercise, assume that the values provided for the year and day are correct, but ensure that the month value is in the range 1–12; if it isn't, set the month to 1.

Provide a `set` and a `get` function for each data member.

Provide a member function `formatDate` that return the date as `string` with the month, day and year separated by forward slashes (/).

Write a test program that demonstrates class `Date`'s capabilities. Alternatively, you can use the `main()` function below.

```
int main() {
    Date d1(19, 1, 2024);
    cout << d1.formatDate() << endl; // should print 19/1/2024
    d1.setDay(17);
    cout << d1.formatDate() << endl; // should print 17/1/2024
    d1.setMonth(5);
    cout << d1.formatDate() << endl; // should print 17/5/2024

    Date d2(29, 13, 2024); // should set month to 1
    cout << d2.formatDate() << endl; // should print 29/1/2024

    return 0;
}
```

Exercise 2

Define the `WareHouse` class, which represents a warehouse. The class should contain two vectors as private members, with the first containing the products' codes and the second one their corresponding quantities. The class should contain the following functions:

- `void stock(int c , int q)` which adds the code `c` in the codes vector only if it is not already stored and the quantity `q` in the quantities vector. If the code is already stored, the function should add the quantity `q` to the existing quantity.
- `void order(int c, int q)` which corresponds to an order for the product with code `c`. The function should check if the code is valid and if there is quantity available and, if so, subtract `q` from it, otherwise a related message should be displayed.
- `void show(int c)` which checks if the code `c` is valid, and if yes it displays the available quantity for the respective product, otherwise a related message.
- The default constructor.

Write a program that tests the operation of the class. Alternatively, you can use the `main()` function below.

```
int main() {
    WareHouse w;
    w.stock(1, 10); // print "10 items with code 1 added"
    w.stock(2, 20); // print "20 items with code 2 added"
    w.stock(3, 30); // print "30 items with code 3 added"
    w.stock(1, 5);  // print "5 more items with code 1 added, total 15"
    w.order(1, 3);  // print "3 items delivered with code 1"
    w.order(2, 25); // print "the requested quantity is not available"
    w.order(2, 11); // print "11 items delivered with code 2"
    w.order(4, 5);  // print "Code 4 not found"
    w.show(1);      // print "12 items are stored with code 1"
    w.show(2);      // print "9 items are stored with code 2"
    w.show(3);      // print "30 items are stored with code 3"
    w.show(4);      // print "Code 4 not found"
    return 0;
}
```

Exercise 3

Define the `Book` class with private members the `title` of the book, its `code`, and its `price`. It should also contain the following public functions:

- `void review(float grd)` which grades the book with the grade `grd`. The `review()` can be called multiple times for the same book. The final score is the average of the grades.
- a constructor that accepts as parameters the data of the book and assigns them to the corresponding members of the object.

Add to the class any other function or member you think is needed.

The following test program declares an array of five objects of type `Book` and initializes it. It calls the `review()` of the objects few times. Then, the program read the code of a book from user and, if the book exists, the program display its score and price. If the book does not exist or it exists but has not been graded, the program display a related message.

```
int main() {
    Book books[5] = {
        Book("The C\texttt{++} Programming Language", 1, 60),
        Book("The Mythical Man-month", 2, 40),
        Book("The Pragmatic Programmer: Your Journey to Mastery", 3, 50),
        Book("The Art of Computer Programming", 4, 50),
        Book("C\texttt{++} For Dummies", 5, 30)
    };
    books[0].review(5);
    books[0].review(4);
    books[1].review(4);
    books[2].review(2);
    books[3].review(5);
    int code;
    cout << "Enter code: ";
    cin >> code;
    for (int i = 0; i < 5; i++) {
        if (books[i].getCode() == code) {
            if (books[i].getScore() == 0) {
                cout << "The book has not been graded yet" << endl;
            } else {
                cout << "The book has score " << books[i].getScore() << " and price " <<
                    books[i].getPrice() << endl;
            }
            return 0;
        }
    }
    cout << "The book does not exist" << endl;
    return 0;
}
```

Exercise 4

Define the class `Polynomial` that simulates a polynomial of the form $f(x) = ax^2 + bx + c$. The class should contain:

(a) The default constructor that initializes the coefficients of the polynomial, that is, `a`, `b`, and `c` to 1. The coefficients should be private members and assume that they are integers.

(b) A constructor that initializes the coefficients of the polynomial with given values.

(c) The `format()` function that returns a string containing the polynomial in the form `ax^2 + bx + c`.

(d) The `eval(int x)` function that returns the result of evaluating the polynomial at `x`.

(e) An overloaded function of the `+` operator, so that additions may be performed between polynomials (e.g., `r=p + q`). That is, the function should return a polynomial with coefficients the sum of the coefficients of `p` and `q`.

(f) An overloaded function of the `[]` operator, so that we can assign new values to the coefficients of the polynomial The signature of the function should be `int& operator[](int index)` which returns a reference to the coefficient of the polynomial at the given index.

E.g., with the statement `poly[0] = 3` the coefficient `a` of the `poly` object should become `3`, while with the statement `poly[1] = 10` the coefficient `b` should become 10.

If the value of the index is not valid, that is, it is not `0`, `1`, or `2`, the program should terminate. (You can use the `exit()` function from `<cstdlib>` to terminate the program.)

(g) An overloaded function of the `==` operator, so that we can check if two polynomials are equal, that is, if their respective coefficients are equal.

Write a program that tests the functionality of the Polynomial class.

Exercise 5

We are prototyping a robot that refills glasses during dinner. Every glass holds 200 milliliters. During dinner, people either drink water or juice, and as soon as there is less than 100 ml left in the glass, the robot refills it back to 200 ml. Create a class Glass with one public int field LiquidLevel and methods public Drink(int milliliters) that takes the amount of liquid that a person drank and public Refill() that refills the glass to be 200 ml full. Both methods should not return any value. Initially set LiquidLevel to 200. In the Main method create an object of class Glass and read commands from the screen until the user terminates the program (see next). Don't forget to refill the glass when needed!

Exercise 6

Create a class called Employee that includes three pieces of information as instance variables—a first name (type String), a last name (type String) and a monthly salary (double). If the monthly salary is not positive, set it to 0.0. Write a test application named EmployeeTest that demonstrates class Employee's capabilities. Create two Employee objects and display each object's yearly salary. Then give each Employee a 10% raise and display each Employee's yearly salary again.

Exercise 7

Create a class called Book to represent a book. A Book should include four pieces of information as instance variables-a book name, an ISBN number, an author name and a publisher. Provide methods (query method) for each instance variable. Inaddition, provide a method named getBookInfo that returns

the description of the book as a String (the description should include all the information about the book). You should use this keyword in member methods and constructor. Write a test application named BookTest to create an array of object for 5 elements for class Book to demonstrate the class Book's capabilities.

Use accessor and mutator in this question.

****